

THEORETISCHE INFORMATIK · GRUNDLAGEN AUFBAUEN

Dein Lernskript für Woche 1

Für Patrik · Klausurvorbereitung zum 20.11.2026

Dieses Skript beginnt bei den Grundlagen. Es erklärt die Fachbegriffe und Zeichen, entwickelt die Verfahren an Beispielen und führt dich anschließend zu den Aufgaben aus deinen Unterlagen. Du musst die Fragen aus dem Standortcheck noch nicht beantworten können.

Was du am Ende verstehen und anwenden kannst

Du kannst Wörter und Sprachen unterscheiden, mit Sprachen rechnen, einfache reguläre Ausdrücke lesen und aufstellen sowie kleine deterministische endliche Automaten konstruieren und ausführen.

Der Stoff dieser Woche

Baustein	Inhalt
Grundidee	Probleme, Eingabegröße, Aufwand und Motivation formaler Sprachen
Grundlagen	Mengen, Alphabet, Wort, ϵ , Σ^* , Sprache und Grammatik
Rechnen mit Sprachen	Schnitt, Vereinigung, Differenz, Komplement, Verkettung, Abschluss und Rechtsquotient
Reguläre Sprachen	RegEx, Vorrangregeln, Beispiele und Verbindung zum Automaten
Automaten	DEA, Übergänge, Endzustände, Fehlerzustand und Komplement
Anwendung	Aufgaben 1-4 erklärt; zusätzliche Übungen und Lösungen

Die Kapitel zu NEA, Potenzmengenkonstruktion und Minimierung folgen in Woche 2. Die Chomsky-Hierarchie wird hier zur Orientierung eingeführt; ihre Beweise und weiterführenden Modelle kommen später.

ORIENTIERUNG

So arbeitest du mit dem Skript

Lies zuerst die Erklärung und rechne die Beispiele mit. Decke danach die Lösung ab und versuche eine ähnliche Aufgabe. Die Übungen am Ende dienen erst dann als Kontrolle. Du musst nicht das ganze Skript an einem Tag durcharbeiten.

Lerneinheit	Lese- und Arbeitsauftrag
1 · Grundlagen S. 3-8	Motivation und Komplexität; Zeichen und Mengen; Alphabet, Wort und ϵ . Erkläre jedes neue Zeichen an einem eigenen Beispiel.
2 · Sprachen S. 9-14, 31	Σ^* , formale Sprachen und Sprachoperationen; danach Aufgabe 2 und die Rechenbeispiele aus der Einführung.
3 · RegEx S. 17-20, 28	Reguläre Ausdrücke schrittweise lesen und selbst erstellen; danach Aufgabe 3a-c.
4 · DEA S. 21-27	Automaten lesen und ausführen; Beispiele nachvollziehen; Aufgabe 4a und 4c bearbeiten.
5 · Verknüpfen S. 15-16, 28-33	Grammatiken und Hierarchie einordnen; Aufgabe 1 als Schaubild; Aufgabe 3d und 4b/d besprechen.
6 · Wiederholen S. 34-41	Zusätzliche Übungen, Lösungen und Wochencheck. Unsichere Inhalte erneut an einer veränderten Aufgabe bearbeiten.

Wo du welche Themen findest

Das Skript folgt genau dieser Reihenfolge: **Verstehen** → **Beispiele** → **Originalaufgaben** → **selbst üben** → **Lösungen** → **Kurzüberblick**. Die PDF-Lesezeichen führen direkt zu den einzelnen Abschnitten.

Drei Fragen bei jedem Beispiel

1. Was ist gegeben?
2. Welche Regel wird angewendet?
3. Warum erfüllt das Ergebnis genau die geforderte Bedingung?

Quellenkürzel: [E] Einführung; [R] Reguläre Sprachen 02_01; [A] Aufgabenblatt; [L] Aufgabenlösungen; [C] CheatSheet; [M] Masterplan. Die genauen Dateinamen stehen im Quellenverzeichnis. Alle Quellenangaben nennen PDF-Seiten. Erklärttexte, zusätzliche Beispiele und Zeichnungen sind für dieses Lernskript neu ausgearbeitet.

01 · MOTIVATION

Worum geht es überhaupt?

Theoretische Informatik untersucht, **was Computer grundsätzlich berechnen können, wie Probleme beschrieben werden und welchen Aufwand eine Lösung benötigt**. Dazu blendet man konkrete Geräte und Programmiersprachen oft aus und arbeitet mit präzisen Modellen.

Drei Fragen, die du auseinanderhalten solltest

Frage	Beispiel
Kann man das Problem exakt beschreiben?	Welche Zeichenfolge gilt als gültiger Bezeichner oder korrekt aufgebauter Ausdruck?
Gibt es überhaupt ein Verfahren?	Kann ein Programm eine gewünschte Ja/Nein-Frage für jede mögliche Eingabe zuverlässig entscheiden?
Wie aufwendig ist das Verfahren?	Wie steigen Rechenzeit und Speicherbedarf, wenn Eingaben größer werden?

Ein **Problem** ist eine allgemeine Aufgabenstellung. Eine **Instanz** ist ein konkreter Fall. „Sortiere Zahlenlisten“ ist ein Problem; die Liste [7, 2, 5] ist eine Instanz. Ein **Algorithmus** ist ein genau beschriebenes Verfahren zur Lösung solcher Fälle.

Warum sprechen Informatiker über Sprachen?

Viele Eingaben sind Zeichenfolgen: Quellcode, Nummern, Passwörter oder Nachrichten eines Protokolls. Statt jede gültige Eingabe einzeln aufzuzählen, beschreibt man die Menge aller erlaubten Eingaben. Diese Menge heißt **formale Sprache**.

Bei einer Grammatik legt man Regeln fest, mit denen gültige Wörter erzeugt werden. Bei einem Automaten beschreibt man, wie eine Eingabe geprüft wird. Reguläre Ausdrücke beschreiben bestimmte Muster kompakt. Alle drei Sichtweisen treffen später zusammen.

Syntax und Bedeutung

Syntax betrifft den Aufbau. Eine Nummer kann dem geforderten Muster entsprechen, ohne tatsächlich vergeben zu sein. Ebenso kann ein Satz grammatisch aufgebaut sein, ohne inhaltlich sinnvoll zu sein. Eine reine Musterprüfung bestätigt zunächst nur den beschriebenen Aufbau.

01 · EINGABEGRÖSSE UND AUFWAND

Komplexität verständlich erklärt

Die **Eingabegröße n** beschreibt, wie groß der zu bearbeitende Fall ist: etwa die Zahl der Listenelemente, die Länge eines Wortes oder die Anzahl der Bits einer Zahl. Die Laufzeit wird als Funktion dieser Größe betrachtet.

Wachstum	Bei $n = 10$	Bei $n = 100$	Wenn n verzehnfacht wird
n	10	100	10-facher Aufwand
n^2	100	10.000	100-facher Aufwand
2^n	1.024	ca. $1,27 \cdot 10^{30}$	extrem starker Anstieg

Die Tabelle verwendet vereinfachte, **exakte Schrittzahlen**. Die O-Notation beschreibt dagegen eine asymptotische obere Schranke: konstante Faktoren und kleinere Zusatzterme werden für große n ausgeblendet. Zum Beispiel gilt für $T(n) = 3n + 7$: $T(n) \in O(n)$.

Anschauliche Beispiele

- **$O(1)$** : Zugriff auf ein Arrayelement über einen bekannten Index im üblichen Rechenmodell.
- **$O(n)$** : Jedes Zeichen eines Wortes einmal lesen.
- **$O(n^2)$** : Jedes Element mit jedem anderen vergleichen; etwa zwei vollständig durchlaufene, ineinander geschachtelte Schleifen über n Elemente.

Eine Schleife ist nicht automatisch linear und zwei Schleifen sind nicht automatisch quadratisch. Entscheidend ist, **wie oft ihr Inhalt tatsächlich ausgeführt wird**. Ein bekanntes Verfahren kann außerdem je nach Eingabe unterschiedlich lange brauchen; daher unterscheidet man beispielsweise besten und schlechtesten Fall.

Zeit und Speicher sind zwei verschiedene Größen

Ein DEA kann ein Wort Zeichen für Zeichen lesen. Bei einem festen Automaten benötigt er linear viele Übergänge und nur den aktuellen Zustand als zusätzlichen Speicher. Die Eingabe kann länger werden, ohne dass mehr Zustände hinzukommen.

Bei Zahlen ist der Zahlenwert nicht dasselbe wie die Eingabelänge. Die Binärzahl 1000 hat den Wert 8 und die Länge 4. Die vollständige Theorie zu P, NP und Reduktionen gehört zu späteren Wochen.

02 · NOTATION

Die mathematischen Zeichen lesen

Du musst mathematische Schreibweisen wie kurze Sätze lesen können. Die wichtigsten Zeichen lassen sich an kleinen Mengen und Wörtern verstehen.

Zeichen	So liest du es	Beispiel
$\{ \dots \}$	Menge der angegebenen Elemente	$\{0,1\}$ enthält die Zeichen 0 und 1.
$\in / \notin$	ist Element von / ist kein Element von	$0 \in \{0,1\}$; $2 \notin \{0,1\}$.
$\subseteq$	ist Teilmenge von	$\{0\} \subseteq \{0,1\}$; auch $\{0,1\} \subseteq \{0,1\}$.
$\subsetneq$	ist echte Teilmenge von	$\{0\} \subsetneq \{0,1\}$; mindestens ein Element fehlt.
$\emptyset$	leere Menge	$\emptyset$ enthält kein einziges Element.
$ A $	Anzahl der Elemente von A	$ \{0,1\} = 2$.
$ w $	Länge des Wortes w	$ 010 = 3$.
$\cup / \cap$	Vereinigung / Schnitt	Zusammenfassen / gemeinsame Elemente.
$\setminus$	Mengendifferenz	$\{0,1\} \setminus \{1\} = \{0\}$.
A^c	Komplement bezüglich einer Grundmenge	Alles in der Grundmenge, das nicht in A liegt.
$\times$	kartesisches Produkt	$\{0,1\} \times \{a\} = \{(0,a),(1,a)\}$.
$:=$	wird definiert als	$\Sigma := \{0,1\}$ legt das Alphabet fest.

Element und Teilmenge sind verschiedene Aussagen

Bei $L = \{0,10\}$ gilt $10 \in L$. Dagegen gilt $\{10\} \subseteq L$. Ein Wort ist hier ein Element der Sprache; eine Menge von Wörtern kann eine Teilmenge sein.

Die Schreibweise $\subset$ wird in der Literatur uneinheitlich verwendet. Dieses Skript nutzt $\subseteq$ für „Teilmenge, eventuell gleich“ und $\subsetneq$ für „echte Teilmenge“.

02 · NOTATION

Logik und Mengen ohne Vorwissen

Zeichen	Bedeutung	Beispiel in Worten
$\neg$	nicht	$\neg P$: Die Aussage P gilt nicht.
$\wedge$	und	$P \wedge Q$: Beide Aussagen gelten.
$\vee$	oder, einschließlich beider	$P \vee Q$: Mindestens eine gilt.
$\forall$	für alle	$\forall w \in L$: für jedes Wort der Sprache.
$\exists$	es gibt mindestens ein	$\exists w \in L$: mindestens ein solches Wort existiert.
$\exists!$	es gibt genau ein	Es existiert ein einziges solches Element.
$\Rightarrow$	daraus folgt	$P \Rightarrow Q$: Wenn P gilt, dann gilt Q.
$\Leftrightarrow$	genau dann, wenn	P und Q sind gleichbedeutend; beide Richtungen gelten.
$\mathbb{N}_0$	natürliche Zahlen mit 0	$\{0,1,2,3,\dots\}$; $n \geq 0$ erlaubt $n = 0$.
$\{x \in M : P(x)\}$	alle x aus M, für die P gilt	$\{n \in \mathbb{N}_0 : n < 3\} = \{0,1,2\}$.

Eine Menge speichert keine Reihenfolge und keine Wiederholungen

Es gilt $\{0,1\} = \{1,0\}$ und $\{0,0,1\} = \{0,1\}$. Deshalb hat die zuletzt genannte Menge zwei Elemente. Ein Wort ist dagegen eine **geordnete Folge**: 01 und 10 sind verschiedene Wörter.

Was bedeutet „genau dann, wenn“ für einen Beweis?

Bei „regulär genau dann, wenn durch einen DEA erkennbar“ brauchst du zwei Richtungen: Aus regulär muss DEA-erkennbar folgen. Zusätzlich muss aus DEA-erkennbar regulär folgen. Nur eine Richtung genügt nicht.

Begriffe in einen Satz übersetzen

$w \in L \Leftrightarrow |w| = 2$ bedeutet: „Das Wort w gehört genau dann zu L , wenn seine Länge 2 ist.“
Der betrachtete Wortbereich muss dabei festgelegt sein, etwa $w \in \{0,1\}^*$.

03 · DIE BAUSTEINE

Alphabet und Wort

Ein **Alphabet Σ** ist eine endliche Menge erlaubter Symbole. Das große griechische Sigma wird „Sigma“ ausgesprochen. Beispiele sind $\Sigma = \{0,1\}$, $\Sigma = \{a,b,c\}$ oder eine festgelegte Menge von Tastaturzeichen.

Ein **Wort w über Σ** ist eine endliche Folge von Symbolen aus Σ . In der theoretischen Informatik braucht ein Wort keine Bedeutung im Alltag. Über $\{0,1\}$ sind 0, 101 und 00001 Wörter. Die Folge 102 ist kein Wort über diesem Alphabet, weil 2 nicht erlaubt ist.

Begriff	Beispiel	Worauf es ankommt
Alphabet	$\Sigma = \{0,1\}$	Welche Symbole sind erlaubt?
Wort	$w = 010$	In welcher Reihenfolge treten sie auf?
Wortlänge	$ 010 = 3$	Wie viele Symbole stehen in der Folge?
Leeres Wort	ε	Es enthält kein Symbol und hat Länge 0.

Ein Symbol kann mehr als ein gedruckter Buchstabe sein

In den Folien tauchen Alphabete wie $\{\text{Menschen, Leben, Tanzen, Welt}\}$ auf. Wenn diese Einträge als **einzelne Symbole** vereinbart sind, hat die Folge „Menschen · Tanzen“ die Länge 2. Wird dagegen jeder Buchstabe einzeln gelesen, ergibt sich ein anderes Alphabet und eine andere Wortlänge.

Darum muss bei jeder Aufgabe zuerst klar sein, ob Übergänge einzelne Buchstaben oder bereits zusammengefasste Tokens lesen. Die Schreibweise allein entscheidet das nicht; die Definition des Alphabets entscheidet.

Endlich lang, aber ohne feste Obergrenze

In Σ^* ist jedes einzelne Wort endlich. Trotzdem gibt es für ein nichtleeres Alphabet Wörter jeder endlichen Länge. Die Formulierung „Wörter können unendlich lang sein“ auf Einführungsfolie 59 gehört zur gesonderten Theorie unendlicher Wörter und wird hier nicht für Σ^* verwendet.

03 · LEERE UND VERKETTUNG

ϵ , $\emptyset$ und $\{\epsilon\}$ sicher unterscheiden

Ausdruck	Was ist das?	Größe bzw. Länge
ϵ	Epsilon: das leere Wort. Eine Folge mit null Symbolen.	$ \epsilon = 0$
$\emptyset$	Die leere Sprache. Eine Menge ohne Wörter.	$ \emptyset = 0$
$\{\epsilon\}$	Die Sprache mit genau einem Wort: dem leeren Wort.	$ \{\epsilon\} = 1$

Die Zahl 0 als Zeichen ist ebenfalls etwas anderes: Das Wort 0 hat Länge 1. ϵ ist kein zusätzliches Eingabezeichen, das ein Automat lesen müsste.

Konkatenation: Wörter aneinanderhängen

Für $u = 01$ und $v = 110$ bedeutet $u \cdot v$: zuerst u , dann v . Es gilt $u \cdot v = 01110$ und $v \cdot u = 11001$. Der Punkt wird häufig weggelassen. Verkettung ist daher im Allgemeinen **nicht kommutativ**: uv muss nicht gleich vu sein.

$$u\epsilon = \epsilon u = u \quad |uv| = |u| + |v|$$

Das leere Wort verändert beim Anhängen nichts. Die Reihenfolge mehrerer Verkettungen kann geklammert werden, ohne das Ergebnis zu ändern: $(uv)z = u(vz)$. Das heißt **assoziativ**.

Wortpotenzen sind Wiederholungen

Ausdruck für $u = 01$	Ergebnis	Erklärung
u^0	ϵ	Null Wiederholungen.
u^1	01	Eine Wiederholung.
u^2	0101	Zwei Kopien hintereinander.
u^3	010101	Drei Kopien hintereinander.

Die Klammern verraten die Art des Objekts

01 ist ein Wort. $\{01\}$ ist eine Sprache mit einem Wort. $\{0,1\}$ ist eine Menge mit zwei Einträgen. Beim Rechnen zuerst klären, welche Art von Objekt vorliegt.

03 · ALLE MÖGLICHEN WÖRTER

Σ^n , Σ^* und die Potenzmenge

Σ^n ist die Menge aller Wörter mit **genau n Symbolen** aus Σ . Für $\Sigma = \{0,1\}$ ergibt sich:

Ausdruck	Wörter	Anzahl
Σ^0	$\{\epsilon\}$	1
Σ^1	$\{0,1\}$	2
Σ^2	$\{00,01,10,11\}$	4
Σ^3	$\{000,001,010,011,100,101,110,111\}$	8

Bei k Symbolen gibt es für n Stellen k^n Möglichkeiten, weil an jeder Stelle eines der k Symbole stehen darf. Für $n = 0$ wird ausdrücklich $\Sigma^0 = \{\epsilon\}$ festgelegt.

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$$

Der **Kleenesche Abschluss** Σ^* enthält alle endlichen Wörter über Σ , einschließlich ϵ . Für $\Sigma = \{0,1\}$ beginnt er mit $\{\epsilon, 0, 1, 00, 01, 10, 11, \dots\}$. Er enthält unendlich viele Wörter, obwohl jedes davon endlich ist. „Beliebig oft“ bedeutet hier immer eine beliebige **endliche** Zahl von Wiederholungen.

Die Potenzmenge ist etwas anderes

$$P(\{0,1\}) = \{\emptyset, \{0\}, \{1\}, \{0,1\}\}$$

Die Potenzmenge $P(\Sigma)$ enthält alle **Teilmengen des Alphabets**. Ihre Elemente sind Mengen, keine Zeichenfolgen. Bei zwei Alphabetsymbolen besitzt sie $2^2 = 4$ Elemente. Das Wort 001 liegt in Σ^* , aber nicht in $P(\Sigma)$.

Sonderfall: leeres Alphabet

Auch ein leeres Alphabet ist in deinen Folien zugelassen. Dann ist $\emptyset^* = \{\epsilon\}$: Ohne Symbole kann nur das leere Wort entstehen. Hier wird $\emptyset$ als Alphabet aufgefasst; bei der leeren Sprache ergibt ihr Kleene-Stern ebenfalls $\{\epsilon\}$.

04 · SPRACHEN BESCHREIBEN

Was eine formale Sprache ist

$$L \subseteq \Sigma^*$$

Eine formale Sprache L ist irgendeine Menge von Wörtern über dem festgelegten Alphabet. Manche Sprachen lassen sich kurz durch Regeln beschreiben, andere nicht. Eine Beschreibung durch Grammatik oder Automat gehört **nicht** schon zur allgemeinen Definition.

Sprache über $\{0,1\}$	Beispiele und Bedeutung
$L_1 = \{0,10,110\}$	Genau die drei aufgezählten Wörter. 010 gehört nicht dazu.
$L_2 = \{w \in \{0,1\}^* : w \text{ endet auf } 1\}$	Zum Beispiel 1, 01, 101, 111. Nicht: ϵ , 0, 10.
$L_3 = \{w \in \{0,1\}^* : w = 2\}$	Genau $\{00,01,10,11\}$.
$L_4 = \emptyset$	Kein Wort gehört dazu.
$L_5 = \{\epsilon\}$	Nur das leere Wort gehört dazu.

Eine Mengenbeschreibung Schritt für Schritt lesen

$$L = \{u \cdot a \cdot v : u \in \{a,b\}^*, v \in \{a,b\}\}$$

u ist ein beliebig langes, endliches Wort, also auch ϵ . Danach kommt genau ein a . v ist genau ein weiteres Symbol. Damit enthält L genau die Wörter, deren **vorletztes Zeichen** a ist. Beispiele: aa , ab , baa , bab . Das Wort a ist zu kurz.

Anzahl bestimmter Zeichen

Mit $\#_0(w)$ bezeichnen wir die Zahl der Nullen in w ; $\#_1(w)$ zählt die Einsen. So lässt sich die Sprache mit gleich vielen Nullen und Einsen schreiben:

$$L = \{w \in \{0,1\}^* : \#_0(w) = \#_1(w)\}$$

Es gehören ϵ , 01, 10 und 0011 dazu. Die Reihenfolge ist hier nicht festgelegt. Diese Sprache ist daher nicht identisch mit $\{0^n 1^n : n \in \mathbb{N}_0\}$, bei der zuerst alle Nullen und danach alle Einsen kommen.

Das Wortproblem

Zu einer Sprache L und einem konkreten Wort w fragt man: Gilt $w \in L$? Für die Sprache „endet auf 1“ ist die Antwort bei 101 ja und bei 100 nein.

04 · VEREINIGEN, SCHNEIDEN, ERGÄNZEN

Mengenoperationen auf Sprachen

Sprachen sind Mengen. Deshalb funktionieren die üblichen Mengenoperationen auch bei Sprachen. Wir verwenden $A = \{0,01,1\}$ und $B = \{01,10\}$.

Operation	Bedeutung	Ergebnis
$A \cup B$	In A oder in B, auch in beiden.	$\{0,01,1,10\}$
$A \cap B$	In A und zugleich in B.	$\{01\}$
$A \setminus B$	In A, aber nicht in B.	$\{0,1\}$
$B \setminus A$	In B, aber nicht in A.	$\{10\}$

Komplement: Die Grundmenge muss feststehen

$$L^c = \Sigma^* \setminus L$$

Bei Sprachen ist die Grundmenge normalerweise Σ^* . Wenn L alle Binärwörter enthält, die auf 1 enden, enthält L^c alle Binärwörter, die auf 0 enden, **und zusätzlich ϵ** . Das leere Wort endet nicht auf 1.

Mit einem größeren Alphabet ändert sich das Komplement. Deshalb müssen L_1 und L_2 bei den folgenden Regeln relativ zum gleichen Σ^* betrachtet werden.

De-Morgan-Regeln

$$(L_1 \cup L_2)^c = L_1^c \cap L_2^c$$

$$(L_1 \cap L_2)^c = L_1^c \cup L_2^c$$

Die erste Regel sagt: „In keiner der beiden Sprachen“ bedeutet „nicht in L_1 und nicht in L_2 “. Daraus folgt auch $L_1 \cap L_2 = (L_1^c \cup L_2^c)^c$. Diese Form brauchen wir für Aufgabe 3d.

Schnitt ist nicht die Operation aus Aufgabe 2

Der echte Schnitt heißt Durchschnitt und wird mit $\cap$ geschrieben. Die Formel in Aufgabe 2 beschreibt dagegen einen Rechtsquotienten. Dieses Skript trennt die Operationen ausdrücklich.

04 · DAS PRODUKT ZWEIER SPRACHEN

Sprachen miteinander verketteten

$$A \cdot B = \{uv : u \in A \text{ und } v \in B\}$$

Man wählt ein Wort aus A und ein Wort aus B und hängt beide aneinander. Das macht man für alle Paare. Mehrfach entstandene Wörter werden im Ergebnis nur einmal notiert.

Beispiel: $A = \{\epsilon, 0\}$, $B = \{1, 01\}$.

Wort aus A	Wort aus B	Verkettung
ϵ	1	1
ϵ	01	01
0	1	01
0	01	001

$$A \cdot B = \{1, 01, 001\}$$

Es gibt vier Paare, aber nur drei unterschiedliche Ergebniswörter. Deshalb gilt allgemein für endliche Sprachen nur:

$$|A \cdot B| \leq |A| \cdot |B|$$

Gleichheit gilt genau dann, wenn verschiedene Paare stets verschiedene Wörter erzeugen. Bei $A = \{0, 00\}$ und $B = \{1, 11\}$ entstehen tatsächlich vier Wörter: $\{01, 011, 001, 0011\}$. Aus den beiden Anzahlen allein lässt sich die genaue Ergebnisanzahl im Allgemeinen nicht bestimmen.

Die wichtigsten Rechenregeln

Regel	Bedeutung
$A \cdot \{\epsilon\} = \{\epsilon\} \cdot A = A$	Das leere Wort verändert kein Wort.
$A \cdot \emptyset = \emptyset \cdot A = \emptyset$	Ohne ein auswählbares Wort entsteht kein Paar.
$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	Verkettung ist assoziativ.
$A \cdot (B \cup C) = A \cdot B \cup A \cdot C$	Verkettung verteilt sich über Vereinigung.

04 · BELIEBIG OFT ZUSAMMENSETZEN

Potenzen und Abschluss einer Sprache

Wie ein Wort lässt sich auch eine Sprache mehrfach verketteten. Der Unterschied: Bei jedem Faktor darfst du erneut eines ihrer Wörter auswählen.

$$L^0 = \{\varepsilon\} \quad L^1 = L \quad L^2 = L \cdot L$$

Für $L = \{a, bb\}$ gilt $L^2 = \{aa, abb, bba, bbbb\}$. Zwei Faktoren bedeuten also nicht unbedingt zwei Zeichen: Die gewählten Wörter können unterschiedlich lang sein.

Ausdruck	Definition	Beispiel für $L = \{ab\}$
L^*	$L^0 \cup L^1 \cup L^2 \cup \dots$	$\{\varepsilon, ab, abab, ababab, \dots\}$
L^+	$L^1 \cup L^2 \cup L^3 \cup \dots$	$\{ab, abab, ababab, \dots\}$

Der Stern erlaubt **null oder mehr** Wiederholungen. Das hochgestellte Plus verlangt **mindestens eine** Wiederholung. Die Zahl der Wiederholungen ist jeweils endlich. Es gilt $L^+ = L \cdot L^*$ und $L^* = \{\varepsilon\} \cup L^+$.

Die wichtige Ausnahme beim Plus

Ist ε bereits in L , kann schon eine Wiederholung das leere Wort liefern. Für $L = \{\varepsilon, a\}$ gilt deshalb $\varepsilon \in L^+$ und sogar $L^+ = L^* = \{\varepsilon, a, aa, \dots\}$. „Plus bedeutet ohne ε “ ist nur dann richtig, wenn ε nicht in L liegt.

Vier Sonderfälle, die du beherrschen solltest

- $\emptyset^* = \{\varepsilon\}$: Null Wiederholungen sind möglich.
- $\emptyset^+ = \emptyset$: Eine Wiederholung ist nicht möglich.
- $\{\varepsilon\}^* = \{\varepsilon\}$ und $\{\varepsilon\}^+ = \{\varepsilon\}$: Jede Verkettung bleibt leer.

Zwei scheinbar ähnliche Ausdrücke

$\{ab\}^*$ liefert ganze ab-Blöcke: $\varepsilon, ab, abab, \dots$. Dagegen liefert $\{a,b\}^*$ alle Wörter aus a und b , also auch a, ba oder bbb . Die Auswahl eines ganzen Wortes ist etwas anderes als die unabhängige Auswahl einzelner Zeichen.

04 · AUFGABE 2 VOLLSTÄNDIG ERKLÄRT

Rechtsquotient: hinten etwas entfernen

Die Operation des Aufgabenblatts heißt dort irreführend „Durchschnitt“. Die angegebene Formel beschreibt einen **Rechtsquotienten**. Damit keine Verwechslung mit der Mengendifferenz entsteht, schreiben wir ihn hier als A / B :

$$A / B = \{w \in \Sigma^* : \exists z \in B \text{ mit } wz \in A\}$$

Gesucht sind alle Wörter w , die durch Anhängen eines passenden Wortes z aus B zu einem Wort aus A werden. Gleichwertig: Entferne von einem Wort aus A ein **vollständiges, passendes Endstück** aus B . Man darf weder beliebige einzelne Zeichen noch ein Teilstück aus der Mitte entfernen.

Originalaufgabe: $A = \{00,01,001\}$, $B = \{\epsilon,0,1\}$

Wort aus A	Endstück aus B	Übrig bleibt
00	ϵ	00
00	0	0
01	ϵ	01
01	1	0
001	ϵ	001
001	1	00

$$A / B = \{0,00,01,001\}$$

Mehrfach entstandene Wörter werden gestrichen. ϵ entsteht hier nicht, denn dafür müsste ein ganzes Wort aus A auch in B vorkommen. Das ist nicht der Fall. Weil ϵ in B liegt, bleiben alle ursprünglichen Wörter aus A ebenfalls im Ergebnis.

Teil c: Zwei nichtleere Sprachen, leeres Ergebnis

Wähle $A' = \{0\}$ und $B' = \{1\}$. Es gibt kein w , für das $w1 = 0$ gilt. Also ist $A' / B' = \emptyset$, obwohl beide Ausgangssprachen ein Wort enthalten.

Zum direkten Vergleich für die Originalmengen: $A \cap B = \emptyset$, $A \setminus B = A$, aber $A / B = \{0,00,01,001\}$. Alle drei Operationen beantworten verschiedene Fragen.

05 · REGELN UND WORTPROBLEM

Grammatiken erzeugen Wörter

Eine Grammatik beschreibt, wie man gültige Wörter schrittweise erzeugt. Sie arbeitet mit Symbolen, die am Ende im Wort stehen dürfen, und mit Platzhaltern, die während der Erzeugung ersetzt werden.

$$G = (\Sigma, V, S, R)$$

Bestandteil	Bedeutung	Beispiel
Σ	Terminalalphabet: Zeichen des fertigen Wortes.	$\{0,1\}$
V	Endliche Menge von Variablen; disjunkt zu Σ .	$\{S\}$
$S \in V$	Startvariable: Hier beginnt die Ableitung.	S
R	Endliche Menge von Ersetzungsregeln.	$S \rightarrow 0S; S \rightarrow 1$

„Disjunkt“ heißt: $\Sigma \cap V = \emptyset$. Die Variablen sind also keine Terminalzeichen. Eine Regel $l \rightarrow r$ ersetzt eine passende linke Seite l durch r . In allgemeinen Grammatiken muss l mindestens eine Variable enthalten; später schränken wir die zulässigen Regeln weiter ein.

Beispielableitung

$$S \Rightarrow 0S \Rightarrow 00S \Rightarrow 001$$

Zweimal wurde S durch $0S$ ersetzt. Danach wurde S durch 1 ersetzt. Im Ergebnis kommen keine Variablen mehr vor: 001 ist ein erzeugtes Wort. Mit null Anwendungen der ersten Regel entsteht 1 , mit einer Anwendung 01 . Die Sprache lautet:

$$L(G) = \{0^n 1 : n \in \mathbb{N}_0\}$$

S und $00S$ sind Zwischenformen einer Ableitung, keine fertigen Wörter über dem Terminalalphabet. Das Wort 10 lässt sich nicht erzeugen: Eine 1 beendet in dieser Grammatik die Ableitung.

Erzeugen und erkennen

Eine Grammatik erzeugt Wörter durch Regeln. Ein Automat verarbeitet ein vorgegebenes Wort und erkennt, ob es zur Sprache gehört. Der reguläre Ausdruck 0^*1 beschreibt dieselbe Sprache kompakt. Diese drei Beschreibungen vergleichen wir bei regulären Sprachen.

05 · ORIENTIERUNG FÜR DIE NÄCHSTEN WOCHEN

Die Chomsky-Hierarchie einordnen

Nicht jede Sprache lässt sich mit denselben Mitteln beschreiben oder erkennen. Die Chomsky-Hierarchie ordnet Sprachklassen nach den erlaubten Grammatikregeln und den dazu passenden Rechenmodellen.

Typ / Klasse	Grammatikidee	Passendes Modell
3 · regulär	Zum Beispiel rechtslineare Regeln $A \rightarrow aB$ oder $A \rightarrow a$; Regeln für ϵ nach der jeweiligen Konvention.	Endlicher Automat: endlich viele Zustände.
2 · kontextfrei	Links steht genau eine Variable: $A \rightarrow u$.	Nichtdeterministischer Kellerautomat: zusätzliche Stapelstruktur.
1 · kontextsensitiv	Äquivalente Sicht: Regeln verkürzen nicht; Sonderregel für ϵ möglich.	Nichtdeterministische, linear beschränkte Turingmaschine.
0 · allgemein	Keine weiteren Einschränkungen über die allgemeinen Grammatikregeln hinaus.	Turingmaschine als Erkenner; bei Nichtzugehörigkeit muss sie nicht halten.

regulär $\subsetneq$ kontextfrei $\subsetneq$ kontextsensitiv $\subsetneq$ Typ 0

Jede Sprache einer kleineren Klasse gehört auch zu den größeren Klassen. Umgekehrt gibt es jeweils Sprachen, die erst in einer größeren Klasse liegen. Die Symbole A und B in der Tabelle sind Variablen; a ist ein Terminalzeichen; u ist eine geeignete endliche Zeichenfolge.

Warum zusätzlicher Speicher wichtig wird

„Endet auf 1“ lässt sich mit endlich vielen Zuständen prüfen. Bei $\{0^n 1^n : n \in \mathbb{N}_0\}$ muss die Anzahl der Nullen mit einer späteren Anzahl von Einsen verglichen werden, ohne feste Obergrenze. Diese Sprache ist kontextfrei, aber nicht regulär. $\{a^n b^n c^n : n \in \mathbb{N}_0\}$ ist kontextsensitiv, aber nicht kontextfrei.

Was du in Woche 1 daraus mitnehmen sollst

Endlich viele Zustände reichen für viele Muster, aber nicht für jedes Problem. Aus „nicht regulär“ folgt nicht automatisch „kontextfrei“. Die genaue Zuordnung und formale Beweise folgen später. Auch eine Typ-0-Sprache kann entscheidbar sein; nur die gesamte Klasse enthält ebenfalls unentscheidbare Sprachen.

06 · REGULÄRE SPRACHEN UND AUSDRÜCKE

Was „regulär“ genau bedeutet

Eine reguläre Sprache entsteht aus einfachen Grundsprachen durch endlich viele Anwendungen bestimmter Operationen. Die Beschreibung wird dabei nach festen Regeln aufgebaut.

Erlaubter Baustein	Beispiel	Was entsteht?
Ein einzelnes Symbol	$\{a\}$	Genau das Wort a .
Das leere Wort	$\{\varepsilon\}$	Genau das leere Wort.
Die leere Sprache	$\emptyset$	Kein Wort.
Vereinigung	$L_1 \cup L_2$	Auswahl zwischen den beiden Sprachen.
Konkatenation	$L_1 \cdot L_2$	Erst ein Wort aus L_1 , dann eines aus L_2 .
Kleene-Stern	L_1^*	Null oder mehr Wörter aus L_1 hintereinander.

Ein **regulärer Ausdruck**, kurz RegEx, ist die schriftliche Beschreibung eines solchen Aufbaus. Der Ausdruck ist nicht selbst die Sprache: $L(r)$ bezeichnet die Sprache, die der Ausdruck r beschreibt.

$$L(a) = \{a\} \quad L(\varepsilon) = \{\varepsilon\} \quad L(\emptyset) = \emptyset$$

Zum Beispiel beschreibt $r = (a \cup b)^*$ alle Wörter aus a und b . Die zugehörige Sprache ist $L(r) = \{a,b\}^*$. Die Mengenklammern werden in regulären Ausdrücken normalerweise nicht um jedes einzelne Zeichen geschrieben.

Eine reguläre Sprache kann unendlich sein

0^* beschreibt $\{\varepsilon, 0, 00, 000, \dots\}$. Die Zahl der Wörter ist unendlich, die Beschreibung ist aber kurz und endlich. Umgekehrt ist jede endliche Sprache aus endlichen Wörtern regulär: Man kann ihre Wörter einzeln ausschreiben und durch $\cup$ verbinden.

Wichtig beim Begründungsschreiben

Zeige, dass du von den Grundaussdrücken ausgehend nur erlaubte Operationen benutzt. „Sieht regelmäßig aus“ oder „enthält viele ähnliche Wörter“ ist keine mathematische Begründung.

06 · VORRANG UND BEDEUTUNG

RegEx lesen: Klammern entscheiden

In diesem Skript steht $\cup$ für die Alternative; in Programmiersprachen sieht man dafür häufig $|$. Der Punkt für die Verkettung kann weggelassen werden. Das hochgestellte Plus ist die Abkürzung $r^+ = rr^*$.

Vorrangregeln

1. Klammern
2. Stern und hochgestelltes Plus
3. Konkatenation
4. Vereinigung $\cup$

Ausdruck	Bedeutung	Beispiele
01^*	Eine 0, danach beliebig viele 1en.	0, 01, 011
$(01)^*$	Beliebig viele vollständige 01-Blöcke.	ϵ , 01, 0101
$0 \cup 1$	Genau ein Zeichen, entweder 0 oder 1.	0, 1
$(0 \cup 1)^*$	Beliebiges endliches Binärwort.	ϵ , 1, 0010
$(0 \cup 1)^*0 \cup 1$	Ein Binärwort mit letzter 0, oder genau das Wort 1.	0, 10, 110, 1

Die Klammerfalle aus Aufgabe 3

$$((0 \cup 1)^*0) \cup 1 \neq (0 \cup 1)^*(0 \cup 1)$$

Links gehört 01 nicht dazu: Es endet nicht auf 0 und ist nicht das einzelne Wort 1. Rechts gehört 01 dazu: Der erste Teil liefert 0 und der letzte Teil liefert 1. Rechts entstehen alle **nichtleeren** Binärwörter; ϵ gehört zu keinem der beiden Ausdrücke.

Abgleich mit praktischen RegEx

In theoretischen Aufgaben beschreibt ein Ausdruck normalerweise das **ganze Wort**. Ein Suchbefehl in einer Programmiersprache kann dagegen nur einen passenden Teilstring suchen. Außerdem sind praktische Erweiterungen wie Rückverweise nicht Teil der hier definierten regulären Ausdrücke.

Der Stern wirkt auf genau den unmittelbar davorstehenden Ausdruck. Er ist eine einstellige Operation. Auf Folie 22 steht dazu ungenau „zweier regulärer Ausdrücke“.

06 · FÜNF VOLLSTÄNDIG ERKLÄRTE BEISPIELE

RegEx selbst konstruieren

Arbeite von der sprachlichen Beschreibung zu Bausteinen. Prüfe anschließend immer ϵ , ein möglichst kurzes gültiges Wort und ein knapp ungültiges Wort.

1 · Alle Binärwörter, die auf 1 enden

$(0 \cup 1)^*1$

Vorne ist jedes Binärwort erlaubt, hinten muss genau eine 1 stehen. Akzeptiert: 1, 01, 101. Nicht akzeptiert: ϵ , 10.

2 · Alle Binärwörter, deren vorletztes Zeichen 0 ist

$(0 \cup 1)^*0(0 \cup 1)$

Ein beliebiger Anfang, dann die vorgeschriebene 0, dann genau ein letztes Zeichen. Akzeptiert: 00, 01, 101. Nicht akzeptiert: 0, 11. Dies ist das Beispiel aus den Folien 19-23.

3 · Genau eine 1, sonst nur Nullen

0^*10^*

Vor und nach der einen 1 dürfen beliebig viele Nullen stehen. Akzeptiert: 1, 00100. Nicht akzeptiert: 000 und 101. Für „mindestens eine 1“ müsste auch im Rest jede Binärfolge erlaubt sein: $(0 \cup 1)^*1(0 \cup 1)^*$.

4 · Gerade Anzahl von Einsen, beliebig viele Nullen

$0^*(10^*10^*)^*$

Vorne stehen nur Nullen. Jeder wiederholte Block liefert genau zwei Einsen, dazwischen und danach beliebig viele Nullen. Null Einsen ist gerade: Deshalb gehören ϵ und 000 dazu. 101 gehört dazu, 111 nicht.

5 · Höchstens eine 1

$0^*(\epsilon \cup 1)0^*$

In der Mitte steht entweder gar nichts oder eine einzelne 1. So sind ϵ , 000 und 0010 enthalten, aber 101 nicht.

Warum die Konstruktionen funktionieren

Für jeden Ausdruck müssen beide Richtungen stimmen: Jedes erzeugte Wort erfüllt die Bedingung. Und jedes Wort, das die Bedingung erfüllt, lässt sich in die vorgesehenen Blöcke zerlegen.

06 · NUMMERN UND LANGE AUSDRÜCKE

Die RegEx-Beispiele aus den Folien

Nummern nach dem Schema FFF / BBB(B) / UUUU / P

In der Folienaufgabe stehen F, B, U und P für Ziffern. Die Klammer bei BBB(B) bedeutet: Die vierte Ziffer ist optional. Die Leerzeichen dienen hier nur der Lesbarkeit; die eigentliche Eingabe enthält keine Leerzeichen.

$D = (0 \cup 1 \cup 2 \cup 3 \cup 4 \cup 5 \cup 6 \cup 7 \cup 8 \cup 9)$

$DDD/DDD(\epsilon \cup D)/DDDD/D$

Das kleinstmögliche Alphabet besteht aus den zehn Ziffern und /. D ist nur eine Abkürzung für den oben ausgeschriebenen Ausdruck. Die vierte Ziffer im zweiten Block wird entweder weggelassen oder aus D gewählt.

Eingabe	Prüfung
123/456/7890/1	Gültig: Blocklängen 3 / 3 / 4 / 1.
123/4567/7890/1	Gültig: Blocklängen 3 / 4 / 4 / 1.
12/456/7890/1	Ungültig: Erster Block ist zu kurz.
123/45/7890/1	Ungültig: Zweiter Block ist zu kurz.

Der Ausdruck verwendet nur Zeichenalternativen, ϵ und Verkettung. Er ist regulär. Er prüft das **Übungsmuster**; daraus folgen keine Aussage über reale Vergabe, Behörde oder eine mathematisch gültige Prüfziffer.

Ein langer Ausdruck wird in Alternativen zerlegt

$(c^*d)a^+ \cup ((c^*bb)^*c^*d(abcd)^*da)$

Für das Wort cccdda kommt die rechte Alternative infrage. Wähle $(c^*bb)^* = \epsilon$, $c^* = ccc$ und $(abcd)^* = \epsilon$. Übrig bleibt $\epsilon \cdot ccc \cdot d \cdot \epsilon \cdot da = cccdda$. Das Wort gehört also dazu. Die linke Alternative würde nach genau einem d nur noch a zulassen und passt deshalb nicht.

Arbeitsregel für lange Ausdrücke

Zuerst die äußere Alternative wählen. Dann jeden Wiederholungsblock einzeln mit einem konkreten Wort belegen. Schreibe zum Schluss alle Teilwörter in derselben Reihenfolge zusammen.

07 · DETERMINISTISCHE ENDLICHE AUTOMATEN

Ein DEA ist eine Lesemaschine

Ein DEA liest ein Wort von links nach rechts. Er befindet sich immer in genau einem Zustand. Das nächste Eingabezeichen legt eindeutig fest, in welchen Zustand er wechselt. Nach dem letzten Zeichen entscheidet der aktuelle Zustand über die Akzeptanz.

$$A = (Q, \Sigma, \delta, s, F)$$

Teil	Bedeutung	Was du angeben musst
Q	Endliche Zustandsmenge.	Alle Zustände, auch ein benötigter Fehlerzustand.
Σ	Eingabealphabet.	Die erlaubten Eingabezeichen.
δ	Übergangsfunktion $Q \times \Sigma \rightarrow Q$.	Für jeden Zustand und jedes Zeichen genau einen Folgezustand.
$s \in Q$	Startzustand.	Hier beginnt die Verarbeitung.
$F \subseteq Q$	Menge der akzeptierenden Zustände.	Die Endzustände, in denen Wörter akzeptiert werden.

Wie du ein Zustandsdiagramm liest

- Ein Kreis steht für einen Zustand. Ein Pfeil von außen markiert den Start.
- Ein Doppelkreis kennzeichnet einen akzeptierenden Zustand.
- Eine mit 0 beschriftete Kante bedeutet: Beim nächsten Zeichen 0 diesem Pfeil folgen.
- Eine Schleife führt zurück in denselben Zustand. „0, 1“ an einer Kante fasst zwei Übergänge mit gleichem Ziel zusammen.

Ein Endzustand beendet das Lesen nicht

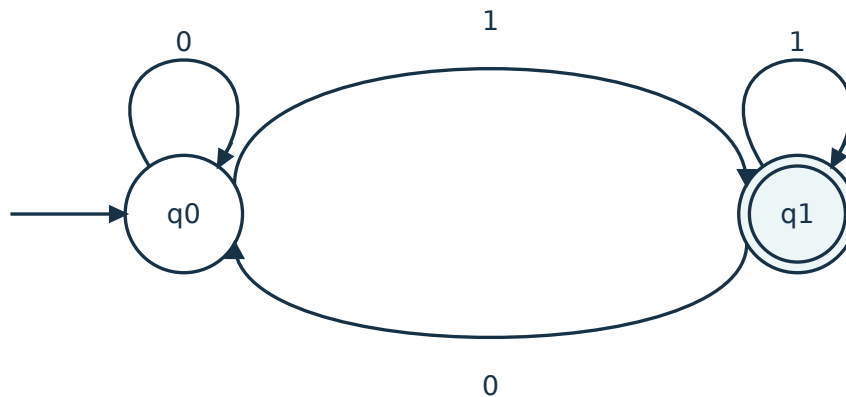
Der Automat kann einen Endzustand während des Lesens wieder verlassen. Entscheidend ist nur der Zustand **nach dem gesamten Wort**. Auch ein nicht akzeptiertes endliches Wort ist nach dem Lesen vollständig verarbeitet.

„Darf sich nichts merken“ ist in den Folien anschaulich verkürzt: Der aktuelle Zustand ist sehr wohl ein begrenztes Gedächtnis. Es gibt aber keinen zusätzlich mit der Eingabelänge wachsenden Speicher.

07 · EIN BEISPIEL GANZ DURCHLAUFEN

Erster DEA: letzte Ziffer ist 1

Die Sprache besteht aus allen Binärwörtern, die auf 1 enden. Wir benötigen nur die Information, ob das bisherige Wort auf 1 endet.



Zustand	Bedeutung	bei 0	bei 1
→ q0	Noch nichts gelesen oder zuletzt 0.	q0	q1
* q1	Zuletzt 1 gelesen.	q0	q1

Formal: $Q = \{q_0, q_1\}$, $\Sigma = \{0, 1\}$, $s = q_0$ und $F = \{q_1\}$. Die Tabelle definiert δ vollständig. → markiert den Start, * einen akzeptierenden Zustand.

Wort 101 verarbeiten

$q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_0 \xrightarrow{1} q_1$

Nach dem ganzen Wort stehen wir in q_1 . Weil $q_1 \in F$ gilt, wird 101 akzeptiert. Das Wort 10 endet dagegen in q_0 und wird nicht akzeptiert.

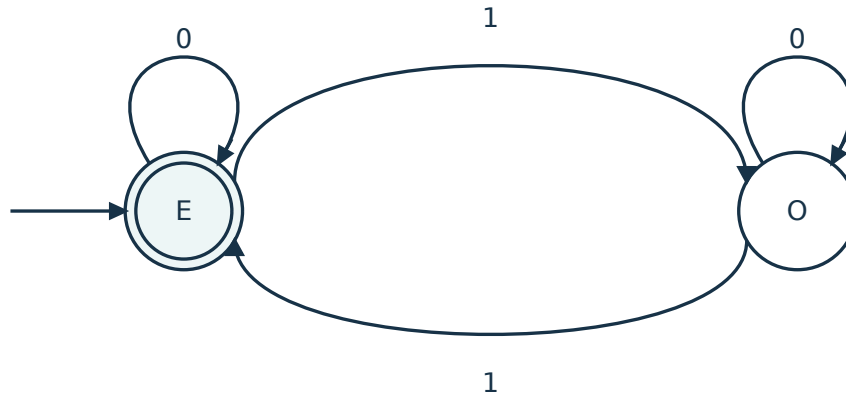
Prüfung des leeren Wortes

Bei ϵ wird kein Übergang ausgeführt. Der Automat bleibt im Startzustand q_0 . Weil dieser nicht akzeptierend ist, gilt $\epsilon \notin L$. Allgemein akzeptiert ein DEA ϵ genau dann, wenn $s \in F$.

07 · ENDLICH VIEL INFORMATION SPEICHERN

Zweiter DEA: gerade Zahl von Einsen

Ein DEA kann nicht beliebig hoch exakt zählen. Er kann aber verfolgen, ob eine Anzahl **gerade oder ungerade** ist. Dafür genügen zwei Zustände, auch bei sehr langen Wörtern.



Zustand	Bedeutung	bei 0	bei 1
$\rightarrow^* E$	Gerade Anzahl von 1en gelesen.	E	O
O	Ungerade Anzahl von 1en gelesen.	O	E

Formal: $Q = \{E, O\}$, $\Sigma = \{0, 1\}$, $s = E$, $F = \{E\}$. Eine 0 verändert die Zahl der Einsen nicht. Jede 1 wechselt zwischen gerade und ungerade.

Wort 1010 verarbeiten

$E \xrightarrow{1} O \xrightarrow{0} O \xrightarrow{1} E \xrightarrow{0} E$

Das Wort hat zwei Einsen und wird akzeptiert. ϵ und 000 werden ebenfalls akzeptiert: Beide enthalten null Einsen, und null ist gerade. Das Wort 111 endet in O und wird abgelehnt.

Was der Zustand aussagt

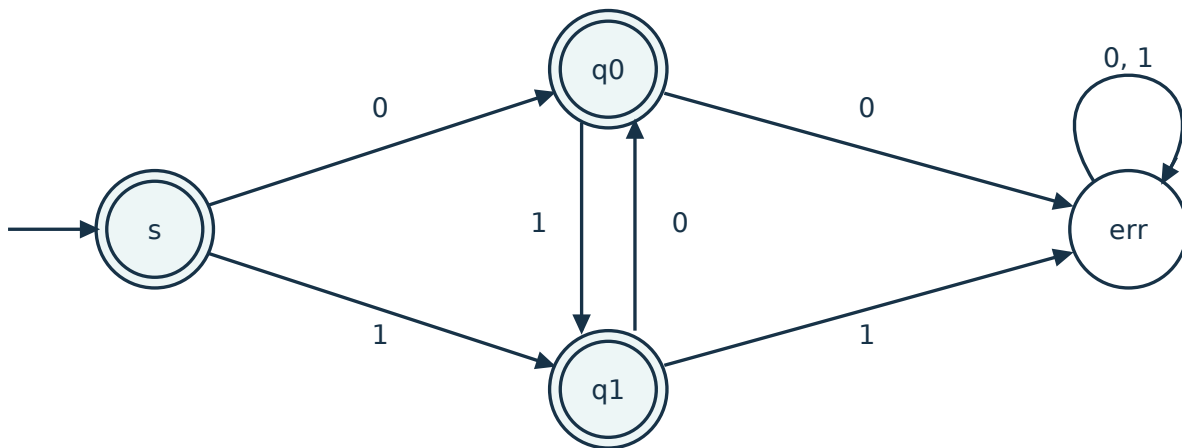
Nach jedem gelesenen Präfix gilt: Zustand E bedeutet gerade, Zustand O ungerade viele Einsen. Eine solche dauerhaft gültige Aussage über die Zustände heißt **Invariante**. Sie erklärt, warum die Konstruktion korrekt ist.

Der passende reguläre Ausdruck lautet $0^*(10^*10^*)^*$. Ein Ausdruck und ein Automat können also dieselbe unendliche Sprache mit endlichen Mitteln beschreiben.

07 · EINEN DEA SELBST ENTWERFEN

Aufgabe 4a: abwechselnde Zeichen

Gesucht sind die Binärwörter, in denen weder 00 noch 11 vorkommt. Man muss sich merken, welches Zeichen zuletzt gelesen wurde. Zusätzlich braucht man den Anfang und einen Zustand für einen bereits festgestellten Verstoß.



Zustand	Bedeutung	bei 0	bei 1
→ * s	Noch kein Zeichen gelesen.	q0	q1
* q0	Bisher gültig, letztes Zeichen 0.	err	q1
* q1	Bisher gültig, letztes Zeichen 1.	q0	err
err	00 oder 11 ist bereits vorgekommen.	err	err

$Q = \{s, q0, q1, err\}$, $\Sigma = \{0, 1\}$, Startzustand s und $F = \{s, q0, q1\}$. Die Zeichnung und die Tabelle geben dieselbe vollständige Übergangsfunktion an.

Beispiele: 0101 endet in $q1$ und wird akzeptiert. 011 endet in err und wird abgelehnt. ϵ wird akzeptiert, denn es enthält keine verbotene Wiederholung. Die Folienzeichnung auf S. 49/50 lässt ϵ durch ihre Endzustandswahl weg; für die wörtliche Aufgabenbedingung gehört es dazu.

So kommst du selbst auf die Zustände

Frage bei einem gelesenen Präfix: Welche Information brauche ich noch für den Rest? Hier reicht „Anfang“, „zuletzt 0“, „zuletzt 1“ oder „bereits ungültig“. Zustände sollen eine Bedeutung haben, bevor du Pfeile zeichnest.

07 · DAS VORLETZTE ZEICHEN

Den Automaten aus den Folien verstehen

Der Automat aus der Vorlesung hat $Q = \{q_0, q_1, q_2, q_3\}$, $\Sigma = \{0, 1\}$, $s = q_0$ und $F = \{q_2, q_3\}$. Seine Übergänge lauten:

Zustand	bei 0	bei 1	Anschauliche Bedeutung
→ q_0	q_0	q_1	Endet auf 00; auch Anfang und einzelnes 0.
q_1	q_2	q_3	Endet auf 01; auch einzelnes 1.
* q_2	q_0	q_1	Endet auf 10.
* q_3	q_2	q_3	Endet auf 11.

Akzeptiert werden die Zustände für 10 und 11. In beiden Fällen ist das **vorletzte Zeichen 1**. Ein Wort muss also mindestens zwei Zeichen lang sein, um akzeptiert werden zu können.

Originalbeispiel 01110

Schritt	Gelesenes Zeichen	Neuer Zustand
Start	Noch keines	q_0
1	0	q_0
2	1	q_1
3	1	q_3
4	1	q_3
5	0	q_2

q_2 ist akzeptierend. Das passt zum Wort 01110: Das vorletzte Zeichen ist 1. Das Wort 101 endet dagegen in q_1 und wird abgelehnt. Dass man unterwegs kurz q_2 erreicht, reicht für eine Annahme nicht aus.

Passender RegEx: $(0 \cup 1)^*1(0 \cup 1)$

Achtung: Zwei unterschiedliche Folienbeispiele

Das RegEx-Beispiel auf S. 19-23 fordert eine **0** als vorletztes Zeichen. Das Automatenbeispiel auf S. 34-42 fordert eine **1**. Es sind zwei verschiedene Sprachen.

07 · SYSTEMATISCH VORGEHEN

Ein Verfahren zum Automatenbau

- 1. Alphabet festlegen.** Welche Zeichen oder Tokens liest der Automat tatsächlich?
- 2. ϵ prüfen.** Erfüllt schon die leere Eingabe die Bedingung? Davon hängt ab, ob der Startzustand akzeptierend ist.
- 3. Notwendige Information bestimmen.** Etwa letztes Zeichen, gerade/ungerade Anzahl oder ein bereits aufgetretener Fehler.
- 4. Zustände benennen und erklären.** Ein Satz pro Zustand verhindert viele falsche Übergänge.
- 5. Für jedes Zeichen einen Übergang eintragen.** Frage, welche Zustandsaussage nach dem Lesen dieses Zeichens gilt.
- 6. Endzustände auswählen.** In welchen Zuständen erfüllt das vollständig gelesene Wort die Bedingung?
- 7. Grenzfälle testen.** ϵ , Wörter mit einem Zeichen, kurze gültige Wörter und beinahe gültige Gegenbeispiele.

Vollständigkeit und Fehlerzustand

In einem vollständigen DEA besitzt jedes Paar aus Zustand und Eingabezeichen genau einen Folgezustand. Eine vereinfachte Zeichnung darf Fehlerkanten auslassen, wenn ein impliziter Fehlerzustand vereinbart ist. Bei Rechnungen, besonders der Komplementbildung, ergänzt du ihn ausdrücklich.

Der Fehlerzustand ϵ besitzt Übergänge für alle Symbole zu sich selbst. Ein bereits festgestellter Verstoß kann in Beispielen wie „kein 00 und kein 11“ nicht durch weitere Zeichen rückgängig gemacht werden.

Warum gleich viele Nullen und Einsen nicht funktioniert

Bei $\{w : \#_0(w) = \#_1(w)\}$ reicht es nicht, nur gerade oder ungerade mitzuzählen. Nach 0^n müsste der Automat eine beliebig große genaue Anzahl speichern, um später genau n Einsen zu verlangen. Endlich viele Zustände können nicht alle diese Anzahlen unterscheiden.

Das ist die anschauliche Begründung zum Folienbeispiel S. 51. Ein formaler Nichtregularitätsbeweis kommt später. Aus „unendlich viele gültige Wörter“ allein folgt die Unmöglichkeit gerade nicht: 0^* ist ein Gegenbeispiel.

08 · ORIGINALAUFGABEN ANWENDEN

Aufgabe 4c: Komplement bilden

Gegeben: $Q = \{q_0, q_1, q_2, q_3, \text{err}\}$, $\Sigma = \{0, 1\}$, $s = q_0$ und $F = \{q_0, q_1\}$. Wir übernehmen die gezeichneten Übergänge und ergänzen die Schleifen bei err:

Zustand	bei 0	bei 1	In F?	In F^c ?
$\rightarrow q_0$	err	q_1	ja	nein
q_1	err	q_3	ja	nein
q_2	q_1	q_0	nein	ja
q_3	q_0	q_2	nein	ja
err	err	err	nein	ja

Die Konstruktion

$$A^c = (Q, \Sigma, \delta, s, Q \setminus F)$$

$$F^c = \{q_2, q_3, \text{err}\}$$

Q , Σ , s und sämtliche Übergänge bleiben gleich. Nur die Menge der akzeptierenden Zustände ändert sich. Vorher akzeptierende Zustände sind danach nicht akzeptierend und umgekehrt. Insbesondere wird **err akzeptierend**: Auch ursprünglich ungültige Wörter gehören zum Komplement.

Wort	Endzustand	Original	Komplement
ϵ	q_0	akzeptiert	abgelehnt
1	q_1	akzeptiert	abgelehnt
0	err	abgelehnt	akzeptiert
11	q_3	abgelehnt	akzeptiert
111	q_2	abgelehnt	akzeptiert

Warum das funktioniert

Jedes Wort aus Σ^* endet in genau einem Zustand. Dieser liegt entweder in F oder in $Q \setminus F$. Deshalb wird jedes Wort in genau einem der beiden Automaten akzeptiert. Beide Automaten verarbeiten jedes endliche Wort vollständig; sie unterscheiden sich in der Annahme, nicht darin, ob sie anhalten.

08 · ENDLICHE SPRACHEN, REGEX UND SCHNITT

Aufgabe 3: die Begründungen ausarbeiten

3a · Jede endliche Sprache aus endlichen Wörtern ist regulär

Sei $L = \{w_1, \dots, w_k\}$. Jedes w_i lässt sich als endliche Verkettung seiner Zeichen schreiben und damit durch einen regulären Ausdruck r_i darstellen. Mit $r = r_1 \cup \dots \cup r_k$ beschreibt man genau L . Auch die Sonderfälle sind abgedeckt: Für das Wort ε verwendet man den Ausdruck ε ; für $L = \emptyset$ den Ausdruck $\emptyset$.

Beispiel: $L = \{0, 01, 110\}$ wird durch $0 \cup 01 \cup 110$ beschrieben. Man braucht dafür keine Wiederholung und keinen Stern.

3b · Die angegebene Sprache richtig lesen

$$\mathbb{L} = (\{0\} \cup \{1\})^* \cdot \{0\} \cup \{1\}$$

Wegen des Vorrangs der Konkatenation enthält $\mathbb{L}$ alle Binärwörter, die auf 0 enden, sowie zusätzlich das einzelne Wort 1. Enthalten: 0, 10, 110, 1. Nicht enthalten: ε , 01, 11.

3c · Regulär und formal?

$$r = (0 \cup 1)^* 0 \cup 1$$

0 und 1 sind Grundausdrücke. Vereinigung ergibt $(0 \cup 1)$; Stern ergibt $(0 \cup 1)^*$; Konkatenation mit 0 und die abschließende Vereinigung mit 1 sind erlaubte Schritte. Also ist r ein regulärer Ausdruck und $\mathbb{L}$ regulär. Weil $\mathbb{L} \subseteq \{0,1\}^*$ gilt, ist $\mathbb{L}$ zugleich eine formale Sprache.

3d · Der Schnitt zweier regulärer Sprachen ist regulär

$$L_1 \cap L_2 = (L_1^c \cup L_2^c)^c$$

Reguläre Sprachen sind unter Komplement abgeschlossen: Man kann einen vollständigen DEA komplementieren. Sie sind auch unter Vereinigung abgeschlossen. Also sind nacheinander L_1^c , L_2^c , ihre Vereinigung und deren Komplement wieder regulär. Nach De Morgan ist die zuletzt entstandene Sprache genau $L_1 \cap L_2$.

Was „abgeschlossen“ bedeutet

Wenn man die Operation auf Sprachen einer Klasse anwendet, bleibt das Ergebnis in derselben Klasse. Bei regulären Sprachen gilt das hier für Vereinigung, Schnitt, Komplement, Differenz, Konkatenation und Kleene-Stern. Diese Aussage darf nicht ungeprüft auf andere Sprachklassen übertragen werden.

08 · FORMULIERUNGEN UND ÄQUIVALENZ

Aufgabe 4b und 4d richtig einordnen

4b · Was verlangt die Primzahl-Aufgabe tatsächlich?

Wörtlich wird ein DEA verlangt, der bei Eingabe einer binär kodierten Primzahl in einem Endzustand endet. Es wird an dieser Stelle nicht ausdrücklich verlangt, dass alle Nichtprimzahlen abgelehnt werden. Genau diese Lücke nutzt die Musterlösung.

$$Q = \{q\}, \Sigma = \{0,1\}, s = q, F = \{q\}$$

$$\delta(q,0) = q \text{ und } \delta(q,1) = q$$

Dieser Ein-Zustands-Automat akzeptiert jedes Binärwort, also insbesondere jede Primzahlkodierung. Er erfüllt die **schwache wörtliche Forderung**, ist aber kein Primzahlprüfer: Er akzeptiert auch 100, die Kodierung von 4, und sogar ϵ .

Die stärkere Forderung „akzeptiert genau die Binärkodierungen aller Primzahlen“ lässt sich bei unbeschränkter Eingabelänge nicht mit einem DEA erfüllen. Der formale Nichtreguläritätsbeweis geht über Woche 1 hinaus. Bei einer festen oberen Grenze wäre die Sprache endlich und damit regulär.

Was du in deiner Lösung schreiben solltest

Nenne die Formulierungsfrage ausdrücklich. Gib für die wörtliche Forderung den obigen Automaten an und sage dazu, dass er Nichtprimzahlen nicht ausschließt. Verwechsle ihn nicht mit einem Automaten, der die Primzahlssprache erkennt.

4d · „Genau die von einem DEA akzeptierten Sprachen“

Die Aussage des Satzes von Kleene ist eine Äquivalenz. Deshalb sind zwei Richtungen zu zeigen:

1. Regulär $\Rightarrow$ DEA-erkennbar: Zu jedem regulären Ausdruck einen passenden Automaten konstruieren. Ein üblicher Weg führt zuerst zu einem NEA und anschließend per Potenzmengenkonstruktion zu einem DEA.

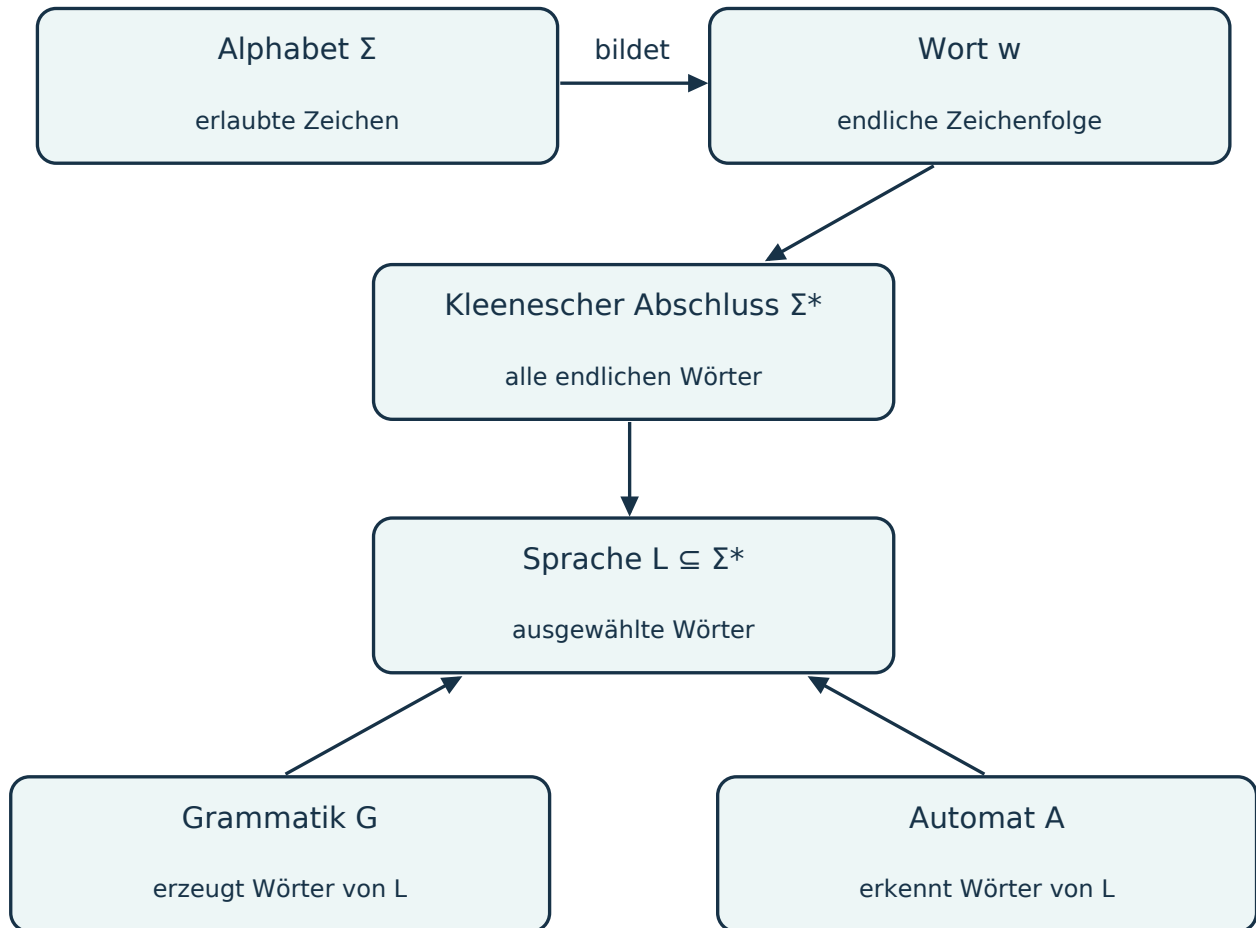
2. DEA-erkennbar $\Rightarrow$ regulär: Zu jedem DEA einen regulären Ausdruck konstruieren, der genau seine Sprache beschreibt; beispielsweise durch systematisches Eliminieren von Zuständen.

Woche 1 verlangt die Idee und die beiden Richtungen. NEA bedeutet nichtdeterministischer endlicher Automat. Die genaue Konstruktion ist Stoff der nächsten Woche. DEA und NEA erkennen dieselbe Klasse regulärer Sprachen.

08 · BEGRIFFE MITEINANDER VERBINDEN

Aufgabe 1: das Gesamtbild

Die Begriffe beschreiben unterschiedliche Ebenen. Das Alphabet liefert die Symbole, Wörter sind Folgen daraus, und eine Sprache wählt bestimmte Wörter aus. Grammatik und Automat bieten dazu zwei verschiedene Arbeitsweisen.



So kannst du das Schaubild erklären

Aus einem Alphabet Σ bilde ich endliche Wörter. Die Gesamtheit aller dieser Wörter heißt Σ^* . Eine formale Sprache L ist eine Teilmenge davon. Ein konkretes Wort w kann zu L gehören oder nicht. Diese Zugehörigkeitsfrage heißt Wortproblem.

Eine Grammatik G erzeugt die Wörter ihrer Sprache $L(G)$. Ein Automat A erkennt die Wörter seiner Sprache $L(A)$. Bei regulären Sprachen kann zusätzlich ein regulärer Ausdruck r dieselbe Sprache beschreiben: $L(G) = L(A) = L(r)$, wenn die Darstellungen passend gewählt sind.

Die Grenze des Bildes

Nicht jede formale Sprache besitzt eine Grammatik oder einen Automaten als Erkennen. Der untere Teil zeigt Beschreibungen dort, wo entsprechende Modelle existieren.

09 · FOLIENBEISPIELE VERVOLLSTÄNDIGEN

Die Rechenaufgaben aus der Einführung

Gegeben sind $L_0 = \{\epsilon, 0, 00, 000\}$ und $L_1 = \{\epsilon, 1, 11\}$. Die kleinstmöglichen Alphabete sind $\Sigma_0 = \{0\}$ und $\Sigma_1 = \{1\}$; für die Verkettung reicht $\Sigma = \{0, 1\}$.

Alle Verkettungen in einer Tabelle

L_0 / L_1	ϵ	1	11
ϵ	ϵ	1	11
0	0	01	011
00	00	001	0011
000	000	0001	00011

$L_0 \cdot L_1 = \{\epsilon, 1, 11, 0, 01, 011, 00, 001, 0011, 000, 0001, 00011\}$

Hier sind alle zwölf Ergebnisse verschieden, also $|L_0 \cdot L_1| = 12$. Die Zahl der Nullen und die Zahl der Einsen bestimmen hier jeweils eindeutig, aus welchem Paar das Wort entstand. Das erklärt, warum in diesem Beispiel $4 \cdot 3$ stimmt.

Die allgemeine Aussage muss eingeschränkt werden

Bei beliebigen endlichen Sprachen können verschiedene Paare dasselbe Wort erzeugen. Dann gilt nur $|A \cdot B| \leq |A| \cdot |B|$. Das Gegenbeispiel $A = \{\epsilon, 0\}$, $B = \{1, 01\}$ aus diesem Skript zeigt den Unterschied. Die allgemeine Gleichheit auf Folie 71 ist daher falsch.

Alle Wörter der Länge 2 über drei Zeichen

$\Sigma = \{0, 1, 2\}$

$\Sigma^2 = \{00, 01, 02, 10, 11, 12, 20, 21, 22\}$

Es gibt drei Möglichkeiten für jede der zwei Stellen, also $3^2 = 9$ Wörter. Für $\{0, 1, 2\}^3$ wären es $3^3 = 27$. Das sind Wortpotenzen des Alphabets, keine Zahlenquadrate der einzelnen Zeichen.

Die weitere Quotientenaufgabe auf S. 72-73 ist bereits bei Aufgabe 2 durchgerechnet. Für beliebiges L gilt außerdem $\emptyset / L = \emptyset$, weil kein ergänztes Wort in der leeren Sprache liegen kann.

09 · DIE ÜBRIGEN DEA-FOLIENBEISPIELE

Endliche Wortlisten erkennen

Beispiel 1: Menschen, Leben, Tanzen, Welt

Die Folien betrachten die vier vollständigen Einträge als einzelne Eingabesymbole. Die Sprache enthält genau je eines dieser Symbole. Ein einfacher DEA hat die Zustände *s*, *f* und *err*: Vom Start *s* führt jedes Symbol nach *f*; von *f* führt jedes weitere Symbol nach *err*; *err* bleibt *err*. Nur *f* ist akzeptierend.

So wird „Menschen“ akzeptiert, die Folge „Menschen · Tanzen“ aber nicht. Werden die Wörter stattdessen als einzelne Buchstaben eingelesen, muss der Automat ihre Buchstabenfolgen verfolgen. Beide Modelle sind möglich, aber die Alphabete unterscheiden sich.

Beispiel 2: {do, double, catch, case} buchstabenweise

Eine nachvollziehbare Konstruktion speichert jeweils den schon gelesenen Anfang eines Zielwortes. Ein solcher Anfang heißt **Präfix**. Das leere Wort ist Präfix jedes Wortes.

Gelesenes Präfix	Fortsetzung	Ergebnis
d	o	do: akzeptierend, aber u darf noch folgen.
do	uble	double: akzeptierend.
ca	tch	catch: akzeptierend.
ca	se	case: akzeptierend.
Ein Präfix	Unpassendes Zeichen	Wechsel nach err.

Formal kann *Q* aus allen Präfixen der vier Wörter und *err* bestehen. *s* ist das leere Präfix. $F = \{\text{do, double, catch, case}\}$. Für ein Präfix *p* und Zeichen *a* gilt $\delta(p, a) = pa$, wenn *pa* wieder ein Präfix eines Zielwortes ist; sonst $\delta(p, a) = \text{err}$. Aus *err* führen alle Übergänge wieder nach *err*.

Warum „do“ nicht sofort beendet

do ist bereits ein gültiges Wort, zugleich aber Präfix von double. Deshalb ist der zugehörige Zustand akzeptierend und hat trotzdem einen sinnvollen ausgehenden Übergang für u. Erst das Ende der Eingabe entscheidet über die Annahme.

Ein mögliches Buchstabenalphabet ist $\{a, b, c, d, e, h, l, o, s, t, u\}$. Die Folien verwenden an dieser Stelle teilweise mehrbuchstabile Tokens. Diese dürfen nicht unbemerkt mit dem hier verwendeten Buchstabenalphabet vermischt werden.

10 · FACHLICH SAUBER LERNEN

Stolperstellen in deinen Unterlagen

Die Unterlagen sind die Grundlage des Skripts. Einige Formulierungen brauchen aber eine Korrektur oder eine ausdrückliche Konvention. Übernimm bei diesen Punkten die folgende präzise Fassung.

Fundstelle	Richtige Einordnung
[E] S. 59: unendlich lange Wörter	Σ^* enthält endliche Wörter. Es gibt beliebig große endliche Längen; unendliche Wörter sind ein anderes Modell.
[E] S. 61; [A] Aufgabe 2: „Durchschnitt“	Die Formel beschreibt den Rechtsquotienten. Schnitt, Differenz und Rechtsquotient sind verschiedene Operationen.
[E] S. 71: $ A \cdot B = A \cdot B $	Allgemein gilt nur $\leq$. Verschiedene Paare können dasselbe Wort erzeugen.
[R] S. 19/21: „unendlich oft“	Bei $*$ wird beliebig oft, aber jeweils endlich oft wiederholt.
[R] S. 22: Stern „zweier“ Ausdrücke	Der Stern ist eine Operation auf einem Ausdruck.
[R] S. 28: Automat merkt sich nichts	Er speichert die begrenzte Information seines aktuellen Zustands.
[R] S. 49/50: Alternation ohne ε	Nach der wörtlichen Bedingung ist ε gültig. Der Start muss dann akzeptierend sein.
[L] S. 5: Komplement und Stoppen	Beide DEAs lesen endliche Wörter vollständig. Geändert wird die Akzeptanz, nicht das Halten.
[C]: positiver Abschluss „ohne“ ε	L^+ enthält ε genau dann, wenn L bereits ε enthält.
[A] 4b; [L] S. 5: Primzahlen	„Alle Primzahlen akzeptieren“ ist schwächer als „genau die Primzahlen akzeptieren“.

Die Chomsky-Tabellen in [E] enthalten zusätzlich stark verkürzte Angaben zur Komplexität. Für diese Woche lernst du daraus die Sprachklassen und Modelle. Die genaue Komplexität eines Wortproblems muss mit dem jeweiligen Eingabe- und Rechenmodell später gesondert behandelt werden.

11 · ÜBUNGSBLATT A

Jetzt selbst üben: Grundlagen

Bearbeite diese Aufgaben erst nach dem erklärenden Teil. Schreibe jeweils den Rechenweg oder einen kurzen Begründungssatz auf. Es ist normal, am Anfang einzelne Regeln nachzuschlagen.

A1 · Alphabet, Wort und Sprache

Gegeben seien $\Sigma = \{a,b\}$ und $L = \{a,ab,ba\}$. Prüfe: $a \in \Sigma$; $ab \in \Sigma$; $ab \in \Sigma^*$; $ab \in L$; $aa \in L$; $\{ab\} \subseteq L$. Erkläre außerdem den Unterschied zwischen $|ab|$ und $|\{ab\}|$.

A2 · Das leere Wort

Bestimme $|\epsilon|$, $|\emptyset|$ und $|\{\epsilon\}|$. Berechne für $K = \{a,bb\}$ die Ausdrücke $K \cdot \{\epsilon\}$, $K \cdot \emptyset$, $\emptyset^*$ und $\{\epsilon\}^+$.

A3 · Alle Wörter einer Länge

Schreibe $\{a,b\}^2$ vollständig auf. Wie viele Wörter liegen in $\{a,b\}^4$? Ist $P(\{a,b\})$ dieselbe Menge wie $\{a,b\}^*$? Begründe mit einem Unterschied.

A4 · Mengenoperationen

$A = \{\epsilon,0,01\}$, $B = \{0,1\}$. Bestimme $A \cup B$, $A \cap B$, $A \setminus B$ und $A \cdot B$. Vergleiche $|A \cdot B|$ mit $|A| \cdot |B|$.

A5 · Rechtsquotient

$A = \{ab,abb,b\}$, $B = \{\epsilon,b\}$. Bestimme A / B . Zeige für jedes Ergebniswort mindestens eine passende Ergänzung z aus B . Gehört ϵ zum Ergebnis?

A6 · Komplexität

Zwei Verfahren benötigen $T_1(n) = 4n + 2$ bzw. $T_2(n) = n^2$ Schritte. Berechne beide für $n = 10$ und $n = 100$. Gib für jedes Verfahren eine passende O-Klasse an und erkläre, warum die Eingabegröße für den Vergleich wichtig ist.

Bevor du die Lösungen öffnest

Prüfe die Art des Ergebnisses: Suchst du ein Wort, eine Menge, eine Zahl oder eine Begründung? Streiche doppelte Wörter aus Mengen. Kontrolliere besonders ϵ und die Reihenfolge bei Verkettungen.

11 · ÜBUNGSBLATT B

Jetzt selbst üben: RegEx und DEA

B1 · Einen Ausdruck lesen

Beschreibe $L(0^*1 \cup \epsilon)$ in eigenen Worten. Entscheide für ϵ , 1, 001, 10 und 11 jeweils, ob das Wort zur Sprache gehört.

B2 · Reguläre Ausdrücke aufstellen

Gib über $\Sigma = \{0,1\}$ je einen regulären Ausdruck an für: a) endet auf 01; b) enthält genau zwei Einsen; c) enthält keine Eins; d) hat genau die Länge 3. Teste jeden Ausdruck mit einem gültigen und einem ungültigen Wort.

B3 · Einen DEA ausführen

Der folgende DEA hat $s = A$ und $F = \{B\}$. Bestimme den Endzustand und die Akzeptanz für ϵ , 0, 01, 10 und 101. Beschreibe anschließend die erkannte Sprache.

Zustand	bei 0	bei 1
$\rightarrow A$	B	A
* B	B	A

B4 · Einen DEA konstruieren

Baue einen vollständigen DEA für alle Binärwörter, die **mindestens eine 1** enthalten. Gib Q , Σ , s , F und die Übergangstabelle an. Begründe, warum ϵ abgelehnt wird. Konstruiere danach seinen Komplementautomaten.

B5 · Fehler finden

Jemand behauptet: „Für das Komplement nehme ich die sichtbaren Doppelkreise weg und mache alle anderen sichtbaren Kreise doppelt. Fehlende Fehlerkanten sind egal.“ Was fehlt? Erkläre außerdem, warum ein Endzustand mitten im Leselauf noch keine endgültige Annahme bedeutet.

B6 · Begriffe verbinden

Gib für die Sprache $\{a,ab\}$ einen regulären Ausdruck an. Erkläre in zwei bis drei Sätzen, wie Alphabet, Σ^* , Sprache, Grammatik und Automat zusammenhängen. Was müsstest du bei einer behaupteten Gleichheit $L(r) = L(A)$ überprüfen?

12 · LÖSUNGEN ZUM ÜBUNGSBLATT A

Lösungen A: nachrechnen und verstehen

A1 · Arten von Objekten

$a \in \Sigma$ ist wahr. $ab \in \Sigma$ ist falsch, weil ab kein einzelnes Alphabetsymbol ist. $ab \in \Sigma^*$ und $ab \in L$ sind wahr. $aa \in L$ ist falsch: Das Wort steht nicht in der aufgezählten Sprache. $\{ab\} \subseteq L$ ist wahr. $|ab| = 2$ ist die Wortlänge; $|\{ab\}| = 1$ ist die Anzahl der Wörter in dieser Menge.

A2 · Leere richtig behandeln

$|\epsilon| = 0$, $|\emptyset| = 0$, $|\{\epsilon\}| = 1$. $K \cdot \{\epsilon\} = \{a, bb\}$; $K \cdot \emptyset = \emptyset$; $\emptyset^* = \{\epsilon\}$; $\{\epsilon\}^+ = \{\epsilon\}$. Beim letzten Ausdruck ist mindestens eine Wiederholung erlaubt, aber das wiederholte Wort ist selbst leer.

A3 · Wortmengen und Potenzmenge

$\{a, b\}^2 = \{aa, ab, ba, bb\}$. Bei Länge 4 gibt es $2^4 = 16$ Wörter. $P(\{a, b\}) = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$ enthält vier Mengen. $\{a, b\}^*$ enthält dagegen unendlich viele endliche Wörter. Beide unterscheiden sich in Elementart und Anzahl.

A4 · Operationen

$$A \cup B = \{\epsilon, 0, 01, 1\}$$

$$A \cap B = \{0\}$$

$$A \setminus B = \{\epsilon, 01\}$$

$$A \cdot B = \{0, 1, 00, 01, 010, 011\}$$

Die Produkte entstehen aus $\epsilon 0$, $\epsilon 1$, 00 , 01 , 010 und 011 . Alle sechs sind verschieden. Hier gilt deshalb $|A \cdot B| = 6 = 3 \cdot 2$; die Gleichheit gilt nicht für alle Sprachen.

A5 · Rechtsquotient

$A / B = \{\epsilon, a, ab, abb, b\}$. Die Ergänzungen sind: $\epsilon \cdot b = b$; $a \cdot b = ab$; $ab \cdot \epsilon = ab$; $abb \cdot \epsilon = abb$; $b \cdot \epsilon = b$. Zusätzlich liefert $ab \cdot b = abb$ denselben Rest ab . ϵ gehört dazu, weil b sowohl in A als auch in B liegt.

A6 · Wachstum

Für $n = 10$: $T_1 = 42$ und $T_2 = 100$. Für $n = 100$: $T_1 = 402$ und $T_2 = 10.000$. Passende Klassen sind $O(n)$ und $O(n^2)$. Bei zehnfacher Eingabe steigt die erste Schrittzahl hier ungefähr um den Faktor 10, die zweite exakt um den Faktor 100. Die feste Zusatzarbeit $+2$ verschwindet in der O -Betrachtung.

12 · LÖSUNGEN ZU B1-B3

RegEx und Automatenläufe

B1 · $0^*1 \cup \epsilon$

Die Sprache enthält beliebig viele Nullen, gefolgt von genau einer 1, und zusätzlich ϵ . Daher sind ϵ , 1 und 001 enthalten. 10 und 11 sind nicht enthalten: Nach der abschließenden 1 darf nichts mehr kommen.

B2 · Vier mögliche Ausdrücke

Bedingung	RegEx	Gültig / ungültig
a) Ende 01	$(0 \cup 1)^*01$	01 / 10
b) Genau zwei Einsen	$0^*10^*10^*$	101 / 111
c) Keine Eins	0^*	ϵ / 1
d) Genau Länge 3	$(0 \cup 1)(0 \cup 1)(0 \cup 1)$	010 / 01

Zu b): Die beiden ausdrücklich hingeschriebenen Einsen sind verpflichtend. Die drei 0^* -Blöcke können nur Nullen beitragen. Daher entstehen genau zwei Einsen. Zu d): Jeder Faktor liefert genau ein Zeichen.

B3 · Lauf des gegebenen Automaten

Wort	Zustandsfolge	Endzustand / Ergebnis
ϵ	A	A / abgelehnt
0	$A \rightarrow B$	B / akzeptiert
01	$A \rightarrow B \rightarrow A$	A / abgelehnt
10	$A \rightarrow A \rightarrow B$	B / akzeptiert
101	$A \rightarrow A \rightarrow B \rightarrow A$	A / abgelehnt

Zustand B bedeutet: Zuletzt wurde 0 gelesen. Zustand A bedeutet: Zuletzt wurde 1 gelesen oder noch kein Zeichen. Daher erkennt der Automat genau die Binärwörter, die auf 0 enden. Ein passender Ausdruck lautet $(0 \cup 1)^*0$.

So korrigierst du einen falschen Lauf

Suche den ersten Schritt, an dem dein Zustand von der Tabelle abweicht. Prüfe genau dieses Paar aus aktuellem Zustand und nächstem Zeichen. Rechne ab dort neu; entscheide die Akzeptanz erst am Schluss.

12 · LÖSUNGEN ZU B4-B6

Lösungen B: Automaten konstruieren

B4 · Mindestens eine 1

Verwende $Q = \{N, J\}$, $\Sigma = \{0, 1\}$, $s = N$ und $F = \{J\}$. N bedeutet „noch keine 1 gesehen“, J bedeutet „bereits mindestens eine 1 gesehen“.

Zustand	bei 0	bei 1
$\rightarrow N$	N	J
$* J$	J	J

Sobald eine 1 auftaucht, geht der Automat nach J und bleibt dort. ϵ bleibt in N und wird abgelehnt. Für das Komplement ändert man nur die Endzustände: $F^c = \{N\}$. Dann wird genau 0^* akzeptiert, einschließlich ϵ .

B5 · Zwei Denkfehler

Erstens muss der DEA vollständig sein. Fehlende Übergänge werden in einen Fehlerzustand ergänzt, der auf allen Zeichen bei sich bleibt. Erst dann tauscht man die akzeptierenden und nicht akzeptierenden Zustände, einschließlich err . Sonst fehlen im Komplement unter Umständen ganze Gruppen von Wörtern.

Zweitens wird immer das gesamte Wort gelesen. Beim Automaten „endet auf 1“ erreicht das Wort 10 nach dem ersten Zeichen einen Endzustand. Das folgende 0 führt wieder heraus. Deshalb wird 10 am Schluss nicht akzeptiert.

B6 · Eine Sprache, verschiedene Beschreibungen

$r = a \cup ab$ oder gleichwertig $a(\epsilon \cup b)$

Über $\Sigma = \{a, b\}$ enthält Σ^* alle endlichen Wörter; $L = \{a, ab\}$ wählt genau zwei davon aus. Eine passende Grammatik erzeugt diese Wörter, ein passender Automat akzeptiert genau diese Wörter. Für $L(r) = L(A)$ müssen alle von r beschriebenen Wörter durch A akzeptiert werden und alle durch A akzeptierten Wörter von r beschrieben sein.

Beispiele prüfen ist nötig, aber nicht immer ein Beweis

Ein falsch eingeordnetes Wort widerlegt eine behauptete Gleichheit sofort. Viele passende Testwörter allein beweisen die Gleichheit unendlicher Sprachen noch nicht. Dafür braucht man die Begründung anhand des Aufbaus oder der Zustandsbedeutungen.

13 · WISSEN ANWENDEN

Wochencheck: erst nach dem Lernen

Beantworte die zehn Fragen ohne Vorlage in eigenen Worten. Du brauchst keine auswendig gelernten Formulierungen. Prüfe deine Antworten danach auf der nächsten Seite.

1. Ist 010 über dem Alphabet $\{0,1\}$ ein Wort? Ist 012 ebenfalls eines? Begründe.
2. Warum sind $\emptyset$ und $\{\epsilon\}$ unterschiedliche Sprachen?
3. Wie lauten alle Wörter aus $\{0,1\}^2$? Gehört ϵ zu $\{0,1\}^*$?
4. Was ist der Unterschied zwischen $L_1 \cup L_2$ und $L_1 \cdot L_2$?
5. Kann $|L_1 \cdot L_2|$ kleiner sein als $|L_1| \cdot |L_2|$? Nenne ein Beispiel oder die Ursache.
6. Welche Sprache beschreibt $(0 \cup 1)^*1$? Wird ϵ beschrieben?
7. Welche Sprache beschreibt $(01)^*$, und warum gehört 010 nicht dazu?
8. Wann akzeptiert ein DEA ein Wort? Wann akzeptiert er ϵ ?
9. Wie erhältst du zu einem vollständigen DEA den Komplementautomaten?
10. Warum genügt für „gerade Anzahl Einsen“ ein DEA, obwohl die Wortlänge nicht begrenzt ist?

Dein praktischer Nachweis

Konstruiere danach ohne Vorlage zwei DEAs: a) endet auf 1; b) enthält keine zwei gleichen Zeichen direkt hintereinander. Gib jeweils die fünf Bestandteile an und teste ϵ sowie ein gültiges und ein ungültiges nichtleeres Wort.

Dein persönliches Wochenziel

Mindestens 8 von 10 Fragen fachlich richtig erklären und die zwei kleinen Automaten selbst aufbauen. Das ist ein Trainingsziel, keine offizielle Notengrenze. Wenn etwas noch nicht klappt, bearbeite den zugehörigen Abschnitt erneut und probiere eine veränderte Aufgabe.

Ein Fehlerprotokoll kann sehr einfach sein: Aufgabe | mein Fehler | richtige Regel | neue Beispielaufgabe. Wiederhole schwierige Punkte am nächsten Lerntag sowie nach ungefähr drei und sieben Tagen.

13 · ABGLEICH MIT BEGRÜNDUNGEN

Antworten zum Wochencheck

- 1. 010 ja, 012 nein.** In 010 sind nur Symbole aus $\{0,1\}$ enthalten. In 012 kommt die nicht erlaubte 2 vor.
- 2. $\emptyset$ enthält kein Wort, $\{\epsilon\}$ enthält ein Wort.** Das eine enthaltene Wort hat zwar Länge 0, es ist aber trotzdem ein Element der Sprache.
- 3. $\{00,01,10,11\}$; ϵ liegt in $\{0,1\}^*$.** Der Stern schließt null Wiederholungen ausdrücklich ein.
- 4. Vereinigung wählt, Verkettung kombiniert.** In der Vereinigung liegen Wörter aus mindestens einer der Ausgangssprachen. Bei der Verkettung entsteht ein neues Wort durch Hintereinanderhängen je eines Wortes aus beiden Sprachen.
- 5. Ja, bei doppelten Ergebnissen.** Für $L_1 = L_2 = \{\epsilon, 0\}$ gibt es vier Paare, aber nur die drei Wörter $\{\epsilon, 0, 00\}$.
- 6. Alle Binärwörter, die auf 1 enden.** ϵ gehört nicht dazu, weil der letzte Faktor eine verpflichtende 1 liefert.
- 7. Null oder mehr vollständige 01-Blöcke.** 010 endet nach einem zusätzlichen halben Block. Jedes erzeugte Wort hat gerade Länge.
- 8. Wenn der Zustand nach dem gesamten Wort in F liegt.** Bei ϵ gibt es keinen Übergang, also wird ϵ genau dann akzeptiert, wenn der Startzustand in F liegt.
- 9. F durch $Q \setminus F$ ersetzen.** Alphabet, Zustände, Start und vollständige Übergangsfunktion bleiben gleich.
- 10. Weil nur zwei Möglichkeiten unterschieden werden müssen.** Die genaue Zahl der Einsen ist egal. Der Zustand speichert lediglich gerade oder ungerade; eine 1 wechselt diese Information, eine 0 nicht.

Die beiden Automaten des praktischen Nachweises stehen in den Abschnitten „Erster DEA“ und „Aufgabe 4a“. Vergleiche die Zustandsbedeutungen und die Übergänge; andere Zustandsnamen sind selbstverständlich möglich.

Was du anschließend tun kannst

Bei sicheren Grundlagen geht es in Woche 2 mit NEA, Potenzmengenkonstruktion, Minimierung und regulären Grammatiken weiter. Falls Wörter, ϵ oder Sprachoperationen noch unklar sind, festige sie zuerst: Die späteren Verfahren verwenden diese Begriffe ständig.

14 · DIE WICHTIGSTEN REGELN AUF EINER SEITE

Kurzüberblick zum Wiederholen

Thema	Regel oder Merksatz
Alphabet / Wort / Sprache	Σ : endliche Symbolmenge. w : endliche Folge. $L \subseteq \Sigma^*$: Menge von Wörtern.
Leere	$ \epsilon = 0, \emptyset = 0, \{\epsilon\} = 1$.
Wortverkettung	$u\epsilon = \epsilon u = u$; $ uv = u + v $; $uv \neq vu$ ist möglich.
Wortmengen	$\Sigma^0 = \{\epsilon\}$; Σ^n : genau Länge n ; Σ^* : alle endlichen Längen.
Mengenoperationen	u : oder; n : und; $\setminus$: links, aber nicht rechts; $L^c = \Sigma^* \setminus L$.
Sprachverkettung	$A \cdot B = \{uv : u \in A, v \in B\}$; doppelte Ergebnisse nur einmal.
Leere Faktoren	$L \cdot \{\epsilon\} = L$; $L \cdot \emptyset = \emptyset$.
Stern und Plus	L^* = Vereinigung aller L^n mit $n \geq 0$; L^+ mit $n \geq 1$. ϵ kann in L^+ liegen.
Rechtsquotient	$A / B = \{w : \exists z \in B \text{ mit } wz \in A\}$; passendes Endstück entfernen.
RegEx-Bausteine	$\emptyset, \epsilon$, einzelne Zeichen; darauf $\cup$, Verkettung, Stern.
RegEx-Vorrang	Klammern; Stern/Plus; Verkettung; Vereinigung.
DEA	$A = (Q, \Sigma, \delta, s, F)$; pro Zustand und Zeichen genau ein Folgezustand.
Akzeptanz	Nach dem gesamten Wort in F . ϵ wird genau dann akzeptiert, wenn $s \in F$.
Komplement-DEA	Vollständig machen; F durch $Q \setminus F$ ersetzen.
Reguläre Darstellungen	RegEx, DEA, NEA und reguläre Grammatik beschreiben dieselbe Sprachklasse.
Komplexität	Aufwand in Abhängigkeit von der Eingabegröße. Zeit und Speicher unterscheiden.

15 · NACHSCHLAGEN IN DEINEN UNTERLAGEN

Quellen und genaue Stoffabgrenzung

Kürzel / Datei	Verwendeter Bereich
[E] TGI_01_Einfuehrung.pdf	S. 4-49: Motivation und Einordnung; S. 51-68: Grundlagen; S. 70-73: Übungen und Lösungsansätze.
[R] TGI_02_01_RegulaereSprachen.pdf	S. 12-13: Äquivalenzziel; S. 14-25: RegEx; S. 27-51: DEA und Beispiele.
[A] Aufgaben (1).pdf	S. 1-4: Aufgaben 1-4. Aufgabe 4c wurde auch anhand des Zustandsdiagramms ausgewertet.
[L] Aufgaben_Loesungen.pdf	S. 1-5: Lösungen zu Aufgaben 1-4; Fehler und verkürzte Formulierungen sind im Skript erläutert.
[C] CheatSheet.pdf	S. 1: Symbole, Mengen, Logik und Grundnotation.
[M] Masterplan_Theoretische_Informatik_20-11-2026(1).pdf	S. 2-4: Umfang und Lernziele von Woche 1; S. 7-8: Hinweise zu Fehlern und zur Mailauswertung.
TGI_00_Organisatorisches.pdf	S. 10: Hinweis auf gemeinsame Klausur mit IT-Sicherheit; keine aktuelle Bestätigung der Prüfungsbedingungen.

Was dieses Skript abdeckt

Es behandelt den fachlichen Stoff der ersten Lernwoche aus dem Masterplan, einschließlich der dort genannten Aufgaben und der passenden Folienbeispiele. Wiederholte Animationsfolien sind zu einer Erklärung zusammengeführt. Reine Vorstellungs-, Unterhaltungs- und Demonstrationsfolien werden nicht einzeln reproduziert.

Die Skripte zu regulären Sprachen II, kontextfreien und kontextsensitiven Sprachen, NP-Problemen und Entscheidbarkeit sowie die Altklausur gehören zu späteren Lernwochen. Die Hierarchie ist hier ein Überblick; die vollständigen späteren Verfahren und Beweise sind nicht vorweggenommen.

Die Erläuterungen, zusätzlichen Übungen, Zeichnungen und Gegenbeispiele wurden neu für dieses Skript formuliert. Der Text gibt keine Garantie für konkrete Klausuraufgaben. Bearbeitungszeit, zugelassene Hilfsmittel und endgültiger Stoffumfang müssen nach der aktuellen Lehrveranstaltungsregelung geklärt werden.